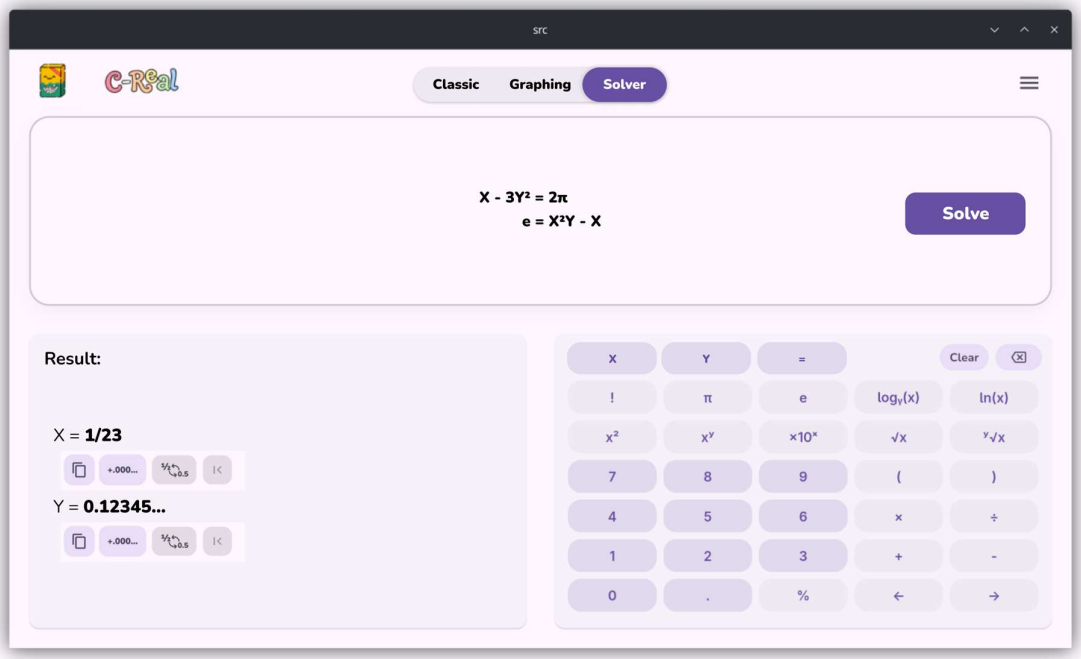
Mockup C-Real 2.0

1. Přídání grafovacího zobrazení



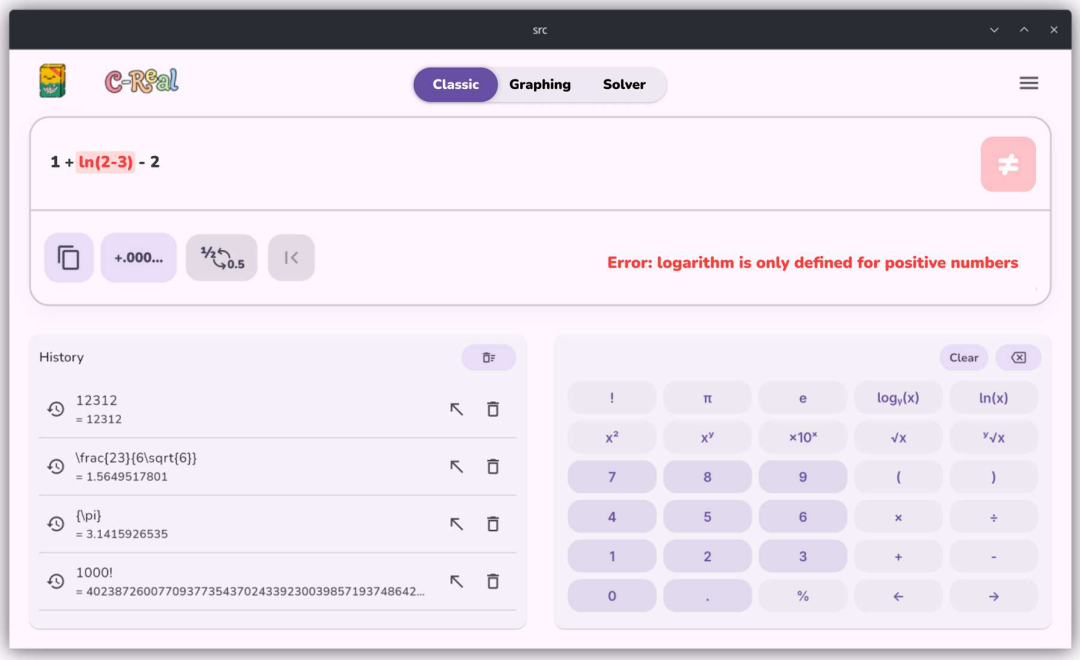
Do horní část UI přibude přepínač módu. Možnosti budou klasická kalkulačka, grafovací zobrazení a rozhraní pro řešení soustav rovnic. Ovládání grafického rozhraní bude zaměřené na jednoduchost a snadnost osvojení. Jednotlivé rovnice budou na klávesnici zadány do vstupu nalevo, odděleny klávesou Enter. Ovládání zobrazení grafu bude přes myš, drag pro posouvání, scroll pro přibližování. Samotné vykreslování bude používat numerické metody, kdy pro každý sloupec pixelů (příp. pro každý N-tý, dle výkonu) bude aproximována hodnota funkce pomocí CRA (constructive real arithmetic) s error bound nastavenou dle rozlišení displeje a úrovně přiblížení. Pro urychlení výpočtu bude rozšířen systém cachování mezivýsledků SyntaxTree pro podobné, ale ne zcela stejné stromy (stejný tvar a literály, ale různé hodnoty x). Omezení tohoto zobrazení bude pouze na funkce (tzn. věci typu $x^2 + y^2 = 1$ nebude schopen vykreslit)

2. Přidání solveru soustav rovnic



Modul solveru soustav rovnic bude přístupné přes přepínač módů v horní části UI a grafické rozhraní bude vzhledově připomínat rozložení klasické kalkulačky. Ze vstupního pole bude odstraněn výsledek a ovládací prvky výsledku. Bude změněno, aby byl obsah zarovnaný na střed, jednotlivé = zarovnané pod sebou. Ovládání bude primárně přes klávesnici, případně vše i přes grafické prvky – tlačítka. Do místa historie se posune zobrazení výsledku jednotlivých neznámých. Každá neznámá bude vypočítána za použití běžných algoritmů, ale pomocí CRA, takže výsledky budou mít také garantovanou přesnost. Tudíž u každé neznáme budou stejné ovládací prvky výsledku, jako v běžné kalkulačce.

3. Názorné zobrazení místa chyby ve vstupním poli



Původně náš stretch goal, ze kterého jsme ale nakonec ustoupili. Umožnoval by uživatelům snáze najít místo, ve kterém došlo ve výpočtu k chybě. Matematická knihovna již tuto funkcionalitu má a na sdíleném syntaktickém stromě správně označuje, kde k chybě konkrétně došlo. Implementace této funkcionality by tudíž znamenalo jí pouze přidat do našeho forku knihovny vykreslující matematické výrazy v LaTeXu.

4. Udělat MathCore multithreaded

Aktuálně aplikace běží na 2 threadech. Jedno pro UI, jedno pro výpočty. Multithreading v MathCore nám umožní výrazně zrychlit běh aplikace a využít celý potenciál procesoru. Toto bude zahrnovat kromě očividného (v binárních operacích každému argumentu přidělit jeho thread z poolu) i další věci, které bude potřeba vyřešit. Hlavní je preprocesor, který před výpočtem projde strom a najde a vypočítá nejvyšší vyžadovanou přesnost jednotlivých transcendentálních literálů (protože ty sdílí mezivýsledky napříč celým programem a výpočet nižší přesnosti, než která byla dříve vypočítána, je prakticky zdarma). Toto navíc umožní vyvarovat se race-condition v cache jednotlivých transcendentálních literálů.